# Synchronization Latency for Databases: PowerSync versus ElectricSQL

## Authors

Aníta Guðrún Finnsdóttir <agif@kth.se>

Olivia Hultman <oliviahu@kth.se>

## Abstract

Sync engines are software component that supports the Seven Ideals for local-first software, which highlight the importance of collaboration between different clients. Previous work on this topic only includes documentation reviews and developer surveys, leading us to the aim of this project: a quantitative experiment comparing the sync engines PowerSync and ElectricSQL. The experiment involves a setup including local and remote databases connected to the sync engines and repeated measurements of the synchronization time, the so-called inconsistency window, under different network latency level settings. The timestamps are collected to form plots visualizing how the sync engines perform in relation to each other, with the network latency also taken into account. This research project can serve as a basis for the selection of the appropriate sync engine, since no empirical, independent, fair study of sync engines has been done before.

## 1 Introduction

### 1.1 Background and Rationale

An alternative to common cloud-based apps, in which data is stored on a remote server, is local-first software. In contrast to cloud-first software, this means the primary storage is hosted locally rather than relying on a local cache for only small parts of the data. A motivation for using the cloud is the collaborative opportunities it offers, but at the same time, it shifts control from the host to remote services. This means that the client depends on being connected to the internet and on abiding by the service's rules and status. By storing the data locally, the client is able to use and manipulate the data despite network and server failures. To combine the benefits of both local and remote storage, Kleppmann et al. [1] define the Seven Ideals that are desirable to achieve for local-first software. This includes short response times, synchronization between clients, offline work capabilities, conflict-free

collaboration, long-lived data, inherent data security, and unrestricted control.

Especially for ideals 2-4, Kleppmann et al. [1] highlight the importance and need for synchronization among different clients to enable collaboration. One way to tick off these three ideals, and possibly a few more, is by using a sync engine. A sync engine has the responsibility to apply changes to all replicas of the data, solve conflicts between client updates, and make sure the storage system eventually converges after a network partition. Shiferaw [2] identifies three types of sync engines available, including relational partial-sync, where you have a local relational database that is partly synchronized with the backend database. Other types include document-based CRDTs and OT-based sync engines that are more focused on collaboration and conflict resolution.

## 1.2 Theory and Related Work

The existing comparative work on local-first sync engines is documentary and perceptual rather than measured. Shiferaw [2] compares six engines against the Seven Ideals using public sources, surveys 12 developers about their experiences, and explicitly identifies controlled laboratory benchmarking as the required next step. Performance figures for relational partial-sync engines are primarily found in vendor documentation.

Measuring the synchronization delay between a write and its visibility to the client is not new. Bermbach and Tai [3] benchmark eventual consistency from the client's perspective in distributed storage systems, where the time between a committed write and the point of its visibility to the reader is referred to as the inconsistency window. This measurement method does not appear to have been applied to local-first sync engines.

Both PowerSync and ElectricSQL fall into the relational partial-sync category described by Shiferaw [2]. They also share the same starting point on the server side: both read the Postgres logical replication stream to pick up changes from the backend database [4] [5]. Where they differ is in how a change reaches application code on the client. ElectricSQL serves shapes, which represent the data to be synchronized, over an HTTP API. Synchronization of a shape occurs in two phases: an initial sync, followed by live mode. The shape logs are sent in the form requested by the client [6], what happens after that is up to the developer. ElectricSQL offers two options for materializing the shapes log: with streaming messages or applying it to a data structure or database [7]. PowerSync uses a different approach where the client keeps a local SQLite database that is automatically kept in sync with the backend database, and all client reads are performed locally in SQLite [8]. This difference matters for measurement. In PowerSync, the point at which a change becomes readable is determined by the engine, when the row is available in SQLite. In ElectricSQL, that point is dependent on what the

developer has built on top of the shape log.

### 1.3 Research Problem, Aim, and Research Questions

Previous work shows that no independent, reproducible, like-for-like measurement of synchronization latency across these engines appears to be available. To address this research gap, this project is focused on experimental measurements to evaluate sync engines. The experiment is narrowed down to test two of the sync engines for relational structural data, namely PowerSync and ElectricSQL, to fit within the scope of the project. The objective is to measure the time it takes for a new write in the backend database to be reflected in the local database on another client, which we refer to as the synchronization latency. This is the same observed value that Bermbach and Tai [3] discuss and refer to as the inconsistency window. To evaluate how this synchronization time would change in a distributed setting that is dependent on an operational network, network latency will be simulated and varied during the experiments. The research questions that should be answered are the following:

- How do PowerSync and ElectricSQL compare in terms of synchronization latency?
- How is the synchronization latency for PowerSync and ElectricSQL affected when the network latency is increased?

## 2 Research Method

### 2.1 Research Design and Methodological Rationale

This project uses a controlled quantitative experiment with two variables, network latency and the sync engine, comparing two options: PowerSync and ElectricSQL. The response variable is synchronization latency: the elapsed time from the commit of a write in the backend Postgres database until that change becomes readable by application code on a separate client. The process is then repeated while adding artificial network latency between that client and the sync service, to see how the two engines react to the added latency. All other elements of the system are held constant across the two sync engines, so that observed differences in latency can be attributed to the engines themselves.

One alternative design that was rejected was a documentary or feature-based comparison. This would mean that official documentation, repositories, and engineering blogs are read and compared. This design had already been done by Shiferaw [2], who scored six engines against Kleppmann's Seven Ideals using public sources, and its design doesn't answer the research questions in this paper. A documentary comparison can only report what the documentation says the engines are built to do. It

cannot produce times in milliseconds, and the times published come from the vendors themselves and are measured differently, so they cannot be compared directly and also have a risk of being unconsciously biased.

Due to the very limited time bound for the project, the experiment will focus on only two sync engines (PowerSync and ElectricSQL), one parameter (network latency), and one metric (synchronization latency). These sync engines were chosen since they are well-known within the field and since they both are used for similar goals. Network latency has a significant impact on the operation of a distributed service and motivates the use of sync engines that support local-first operation, and that's why this was considered an interesting parameter to vary across the experiments. The synchronization latency was chosen as the metric since both engines use the Postgres logical replication stream as their source of change events [4] [5]. This means that the upstream part of the pipeline is architecturally comparable, which makes it more likely that any measured difference comes from the engines' own synchronization paths.

## 2.2 Data, Materials, and System Under Test

Data will be collected during the experiment, with each measurement recorded in a log file as a single row containing the engine, network latency level, run number, sequence number, two timestamps, and the resulting synchronization latency. Repeated measurements are taken for each sync engine, defined by two timestamps, $t_0$ and $t_1$, with the difference between them representing the synchronization latency. $t_0$ is recorded immediately after Postgres confirms the write commit, and $t_1$ is recorded when the new value can actually be read by the application code on the client. For PowerSync, this is when the row can be read from the local SQLite database [8]. For ElectricSQL, where materialization is left to the developer [7], $t_1$ will be recorded after the shape has been applied to a local store, that is, in this case, the local SQLite database. This ensures both engines are measured at the point where data is readable locally, rather than when the change first reaches the client. This definition of $t_1$ is treated as provisional until the pilot test described in Section 3.1 has confirmed that it is comparable across the engines. Both timestamps are taken in the same process on the same machine using the same monotonic clock. This is done to minimize clock synchronization error.

## 2.3 Work Procedure and Data Collection

The sync engines will be deployed in a self-hosted setup so that we can control the whole pipeline, since both engines allow this. ElectricSQL is an Elixir application that connects to Postgres with a connection string, reads the logical replication stream, and serves shapes, which represent the data to

be synchronized, over an HTTP API [4]. PowerSync is available through a Docker image [9], where the client SDKs are released under Apache 2.0, while the server-side PowerSync Service is released under the Functional Source License [10]. The server-side setup will be a Postgres database and the two different sync engines PowerSync and ElectricSQL, each of them deployed using Docker containers. For each of the engines, there will also be a client that is hosted locally. PowerSync has built-in use of a local SQLite database, while ElectricSQL is more open regarding the choice of local storage solution. That makes it possible to use a SQLite database even for the ElectricSQL engine, which makes the results less dependent on the database type. The clients will be very simple applications with only the purpose of sending basic queries to the local database. The database will include just a simple schema and table that the clients target.

The main data collection takes place through a set of identical write sequences for every combination of engine and network latency, so the two engines are compared under a common workload. The network latency is simulated between the client and the sync service, using the Linux tool `tc netem`.To realize these tests, an automated script will be developed that collects the different test runs, with the network latency level varied between the runs. For every latency level, multiple runs will also be performed to avoid temporary external impact on a single result. Each result, in the form of two timestamps representing the start and end of the synchronization together with its metadata, will then be written to a CSV file.

## 2.4 Data Analysis and Quality Assurance

From running the tests, we should obtain a collection of CSV files with measurements in the form of start and stop timestamps for the synchronization, with their corresponding metadata, including the type of sync engine and the current latency level. From these data points, we will use Python and the Matplotlib library to automatically generate plots representing the results.

A cumulative distribution graph of latency at the lowest network latency level will be used to address the first research question. The x-axis will show the latency in milliseconds and the share of measurements below that value on the y-axis, with one curve per engine. If one curve lies to the left of the other one over the whole range, then that engine is faster. If the curves cross, the answer depends on whether typical or slower cases are more important, a point that needs to be analyzed.

The main graph for the second research question will have the added network delay in milliseconds on the x-axis and measured synchronization latency on the y-axis. There will be one line per engine drawn through the medians, with a shaded band indicating the interquartile range. This graph will show whether the two engines degrade in the same way. If both lines rise with roughly the same slope,

then the network affects them equally. If one line rises faster than the other, then the engines respond differently to a worse network.

Since there are many differences in the architectures of PowerSync and ElectricSQL, there could be many potential factors that make the comparison not fair. In addition to this, since the tests are running on a general-purpose computer, there could also be other external factors affecting the performance of the sync engines at the time of measurement. To mitigate this, an identical test environment setup will be used for both of the sync engines, as far as it's possible. To limit the impact of randomness, each test will also be repeated multiple times, and an average result will be obtained. When drawing conclusions and applying this study in a broader perspective, care is needed to ensure that it's clear that the results are only valid under these specific configurations.

To reproduce the study, information about the specific versions of the included components, such as PowerSync, ElectricSQL, Postgres, and SQLite, will be included in the report, as well as the hardware and operating system that act as a testbed. Also, the configuration for each component will be documented, including the network latency levels and the number of repetitions executed. The implementation of the clients, the test script, and the collected measurements will be able to be found in the report or in an open GitHub repository.

### 2.5 Use of AI Tools

We will use AI to gain an overview of a concept or topic, find relevant sources, and help improve the natural and programming language. Grammarly will be used during the writing process, while different AI chatbots will be used for topic or concept outlines, literature identification, and implementation. Even though we plan to use some AI assistance for this project, nothing from it will be taken at face value; we will review and verify every piece of information we use in our research against sources. Our own words exclusively will be included in the report, and the responses from AI chatbots will only be used as a basis to understand the ideas that we are discussing.

## 3 Project Planning and Feasibility

### 3.1 Work Tasks and Milestones

The first work task is to establish the base stack, including Postgres, Docker, and both engines PowerSync and ElectricSQL. For each engine, a server will be connected to the Postgres database, and a client will be implemented as a simple application. For both of the clients, a local SQLite database that

is synced with the central database will be queried, which will require some additional implementation for ElectricSQL. The database and sync engine will be deployed using Docker, while the clients will operate locally. To verify that the engines run and operate as intended, we will conduct a small-scale one-measurement test on one engine first, then repeat the procedure on the other. This milestone is planned for completion by 24 Sept.

Once we have verified the setup is working as intended, we will do a small number of manual measurements on one engine and repeat on the other. What should be measured here is the time from when a new change is committed in the central database until that change is reflected in the local database. The resulting measurements will be compared, and, if necessary, the $t_0$ and $t_1$ measurements will be adjusted and redefined, as the two engines use different synchronization logic. From this, we can begin automating the remaining steps: script execution of tests, including CSV logging of timestamps and a simulated configurable latency level. To ensure the scripts work as intended, short pilot tests will be run to refine the code. This milestone is planned for completion by 28 Sept.

The final task is to run the complete set of measurements on both engines at all latency levels on the same computer. It also needs to be ensured that all other experimental conditions are identical between the tests for the different engines. The results generated in CSV files can then be studied or used for generating plots for further analysis of the results. This is planned for completion by 8 Oct.

## 3.2 Allocation of Responsibilities

The plan is to run all the tests on one computer so the main setup stays on that computer. This requires us to work more together rather than splitting the work, so that we are both familiar with everything. The deployment and execution need to be done on that computer, but the implementation, harness, config, and analysis pipeline can be developed more independently using collaboration tools like GitHub. The initial idea for allocating responsibilities is that deployment and execution will be done on Olivia's computer; therefore, she will have the primary responsibility for that. Olivia will also have primary responsibility for the implementation of the clients, while Aníta will have primary responsibility for the harness and configuration of the setup. The analysis will be done together, as we both must be able to explain and defend the results. Even though we have assigned primary responsibilities, collaborate on those tasks as much as possible so that both of us have a good understanding of the whole project. We will have regular meetings to follow up on the progress of the tasks and make sure that we help each other out if we encounter some problems or difficulties.

### 3.3 Implementational Risks and Mitigation Strategies

One implementation risk for this project is the intention to self-host the servers for both engines. This approach requires additional setup and may introduce complexity. To mitigate this risk, we will begin the server-side setup early and set a firm completion deadline. If server configuration proves more challenging than expected or takes longer than anticipated, we may need to use the cloud version for both engines. Likewise, if we can only self-host one engine, both engines will run on the cloud version to ensure a fair comparison. Falling back to cloud servers introduces additional variables into synchronization latency, such as an uncontrolled WAN segment between the client and the sync server, and the two cloud servers may be hosted in different regions.

Given the time constraints of this course, running all tests for both engines on a single general-purpose computer may take longer than anticipated and could result in incomplete results by the deadline. To ensure a fair comparison, both engines should be run on the same computer to avoid speculation regarding potential hardware or software biases in the tests. This risk can be mitigated by running a pilot test for a fixed duration to estimate total runtime and scheduling overnight tests if the pilot indicates the full test will not finish during normal working hours. It could also be necessary to use a minimal dataset for each engine (e.g., fewer latency levels).

Another risk concerns compatibility between ElectricSQL and PowerSync when measuring $t_1$. Because the two engines use different synchronization mechanisms, with ElectricSQL leaving materialization up to the developer and PowerSync using a preset localized database, inconsistent measurement points could bias the comparison and undermine the construct validity. To mitigate this risk, the definition of $t_1$ in section 2.2 is treated as provisional until both engines have been set up and the pilot test has been conducted, so the measurement points can be confirmed to be aligned across both engines. Even so, the definition of $t_1$ affects more the scope of the claim rather than the validity of the research.

## References

[1] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software,” April 2019, last accessed 3 September 2026. [Online]. Available: https://www.inkandswitch.com/essay/local-first/

[2] E. Shiferaw, “Sync engine comparison for localfirst web applications an empirical study,” Master’s thesis, Tampere University, Faculty of Information Technology and Communication

Sciences, May 2026. [Online]. Available: https://trepo.tuni.fi/bitstream/handle/10024/237992/ShiferawErmyas.pdf?sequence=2

[3] D. Bermbach and S. Tai, “Eventual consistency: How soon is eventual? an evaluation of amazon s3’s consistency behavior,” December 2011, last accessed 11 September 2026. [Online]. Available: https://www.semanticscholar.org/paper/Eventual-consistency:-How-soon-is-eventual-An-of-Bermbach-Tai/ea079e2b3363ab9a8bd84a985f64407be5cf6673

[4] Electric, “Postgres sync,” n.d., last accessed 3 September 2026. [Online]. Available: https://electric.ax/sync/postgres-sync

[5] PowerSync, “Powersync service,” n.d., last accessed 3 September 2026. [Online]. Available: https://docs.powersync.com/architecture/powersync-service

[6] Electric, “Http api,” n.d., last accessed 11 September 2026. [Online]. Available: https://electric.ax/docs/sync/api/http

[7] ——, “Client development,” n.d., last accessed 11 September 2026. [Online]. Available: https://electric.ax/docs/sync/guides/client-development

[8] PowerSync, “Client architecture,” n.d., last accessed 3 September 2026. [Online]. Available: https://docs.powersync.com/architecture/client-architecture

[9] ——, “Self-hosting,” n.d., last accessed 3 September 2026. [Online]. Available: https://docs.powersync.com/intro/self-hosting

[10] ——, “Open-source source-availability,” n.d., last accessed 3 September 2026. [Online]. Available: https://powersync.com/open-source